Object Storage

buckets, keys, whole objects, eleven nines

Watch the 4-min video

In one line: object storage keeps immutable blobs under keys in a flat namespace, served over HTTP, for a few cents per GB-month with eleven nines of durability; keep the bytes there and the facts about them (owner, name, status) in your database.

1 The database that became a file server

Profile photos and videos go into a `bytea` column in the main Postgres database. It works on day one. A year later the database is mostly video: backups take all night, every read replica copies terabytes of files, a restore takes hours, and expensive database disks (provisioned IOPS, replicated) are storing bytes that are read once and never updated. Large binary data has a completely different access pattern from rows: it is big, written once, read whole and streamed. That is exactly what object storage is built for.

2 Buckets, keys and objects

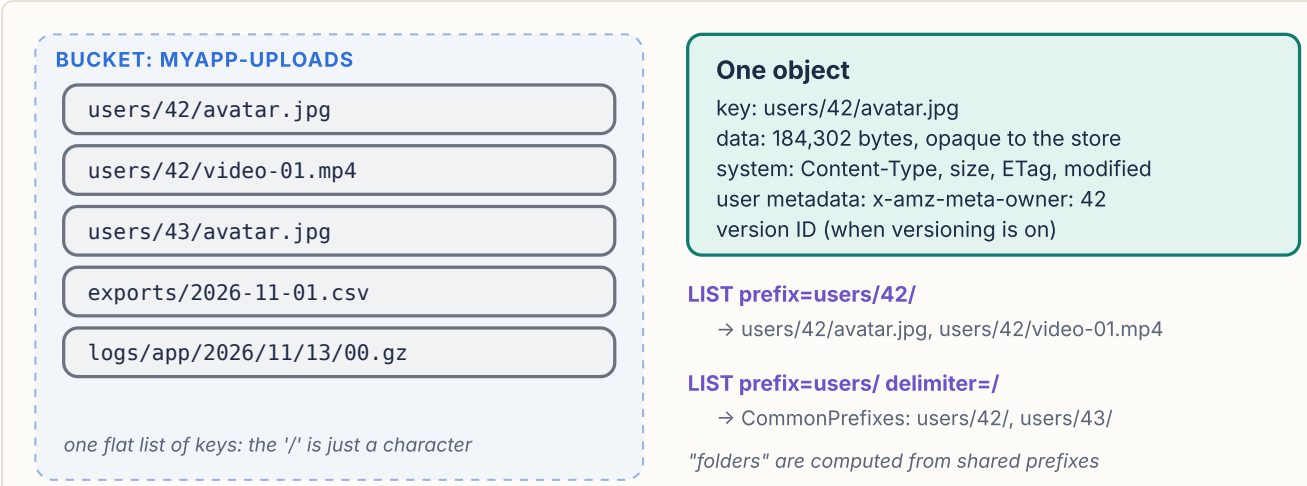


Figure 1. A bucket holds objects under keys. The namespace is flat: `LIST` with a **prefix** and a **delimiter** makes keys look like folders.

- **Bucket** (Azure: *container* inside a *storage account*): a named namespace with its own access policy, region and settings (versioning, lifecycle rules, encryption). S3 bucket names are globally unique.
- **Key**: the object's name, up to 1,024 bytes in S3. `users/42/avatar.jpg` is one string; there is no folder `users/42`. Renaming a "folder" therefore means copying and deleting every object under it.
- **Object**: the data plus metadata. System metadata (Content-Type, Content-Length, ETag, Last-Modified) and a small amount of user metadata (S3: 2 KB of `x-amz-meta-*` headers). Metadata can't be queried: there is no "find all objects owned by user 42".
- **API**: plain HTTP. `PUT`, `GET` (with `Range` for partial reads), `HEAD`, `DELETE`, `LIST` (paged, 1,000 keys per page in S3). Amazon S3, Azure Blob Storage, Google Cloud Storage, and self-hosted MinIO or Ceph RGW all follow this model; many speak the S3 API.

3 Objects are written whole

You can't change byte 40 of an object or append to it (Azure's *append blobs* are a special case). You `PUT` a complete new object, which atomically replaces the old one: a reader sees either the whole old object or the whole new one, never a mix. This immutability is what lets the service replicate and cache objects freely and scale to trillions of them. Reads can still be partial: an HTTP `Range` request fetches any byte range, which is how video players seek and how parallel downloads work.

- **Versioning** keeps every previous version. An overwrite creates a new version; a delete adds a *delete marker* on top, so both are undoable (lesson 3). Old versions are billed, so pair versioning with a lifecycle rule (lesson 12).
- **Concurrent writers**: two PUTs to the same key race and the last one wins; there are no locks. Use unique keys (e.g. a UUID per upload) or conditional requests: S3 supports `If-None-Match: *` (create only if absent) and `If-Match: <ETag>` on writes, and Azure Blob supports ETag conditions and leases.
- **Large objects**: S3 accepts up to 5 GB in a single PUT; bigger files (and anything over ~100 MB, in practice) use multipart upload (lesson 6).

4 Consistency: no longer a gotcha

Until December 2020, S3 was *eventually* consistent for overwrites and listings: you could PUT an object and briefly read the old one, or not see it in a LIST. Systems built workarounds (e.g. a consistent metadata layer). Since then S3 gives **strong read-after-write consistency** for PUT, DELETE and LIST in all regions at no extra cost: once a write returns success, every subsequent read or list sees it. Azure Blob Storage and Google Cloud Storage are also strongly consistent for object operations. What is still *not* consistent: a CDN in front of the bucket (cached copies live until they expire, lesson 11) and cross-region replicas (copied asynchronously, lesson 13).

5 Durability, availability and storage classes

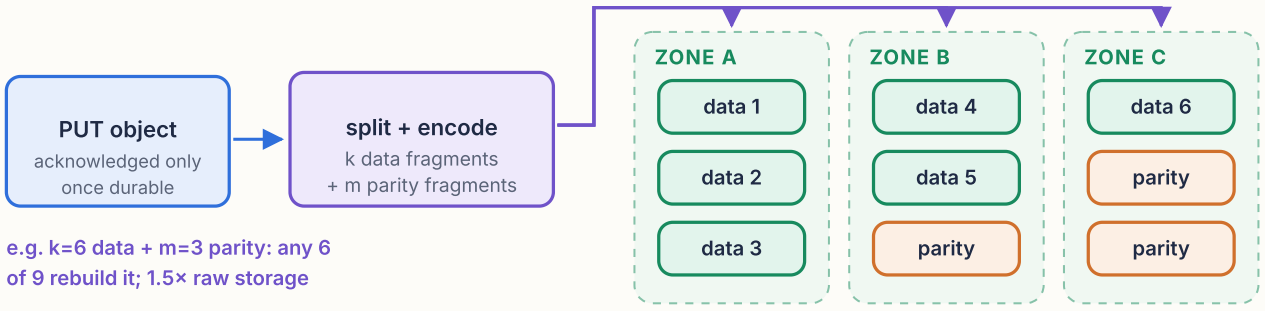


Figure 2. How durability is achieved (conceptually): data is split and encoded, and fragments are spread across failure domains, so losing a disk, a rack or even a zone loses nothing.

Durability is the chance of not losing data; **availability** is the chance a request succeeds right now. S3 Standard is designed for **99.999999999%** (eleven nines) durability by storing data redundantly across at least three Availability Zones, while its availability design target is 99.99%. Eleven nines means that if you store 10 million objects you can expect to lose one about every 10,000 years. Large object stores typically use **erasure coding** rather than full copies: data is split into *k* fragments plus *m* parity fragments, and any *k* rebuild it, so the overhead is far below the 3× of triple replication. Background scrubbing re-checks checksums and repairs fragments.

Durability against hardware is not protection against *you*: a buggy script that deletes objects is executed with eleven-nines reliability. Versioning, Object Lock and replication to another account (lessons 3, 12, 13) cover that.

S3 storage class	First byte	Minimum duration	≈ $/GB-month*	Use for
Standard	milliseconds	none	0.023	active data, user uploads
Standard-IA	milliseconds	30 days, 128 KB min size	0.0125 + per-GB retrieval	backups, older media read occasionally
One Zone-IA	milliseconds	30 days	0.01	re-creatable data (one zone only)
Glacier Instant Retrieval	milliseconds	90 days	0.004	archives read a few times a year
Glacier Flexible Retrieval	minutes to hours (restore first)	90 days	0.0036	archives, compliance copies
Glacier Deep Archive	within 12 h (bulk 48 h)	180 days	0.00099	keep-for-years records
Intelligent-Tiering	milliseconds (default tiers)	none	by tier + small monitoring fee	unknown or changing access patterns

*Approximate us-east-1 list prices for the first 50 TB; they vary by region and change over time. Azure Blob has the equivalent Hot, Cool, Cold and Archive tiers (lesson 3). Requests are billed too: PUT/LIST cost more than GET, and colder tiers charge per GB retrieved.

Lifecycle rules move objects between classes by age or delete them automatically (lesson 12). For comparison, an SSD block volume (lesson 2) costs about 3–4× more per GB than S3 Standard and is tied to one machine; a managed database's storage is usually more expensive still, and then multiplied by every replica.

6 Why not a database, and what you give up

	Relational database	Object storage
Good at	small rows, indexes, queries, transactions	large immutable blobs, streaming reads
Scale	one primary's disk; replicas copy everything	practically unlimited; no capacity planning
Serving files	bytes flow through DB and app servers	clients download directly or via a CDN
Updates	in place, row locks	whole-object replace, no locks
Queries	SQL over any column	GET by key, LIST by prefix only
Transactions	ACID across rows and tables	none across objects

The standard design uses both: **bytes in the bucket, metadata in the database**. A `files` row holds the object key, owner, original file name, size, content type, checksum and status; the bucket holds the content (lesson 7). Because there is no transaction across the two, write the object first and the "ready" row second, and periodically clean up objects that no row points to.

7 Example: user uploads

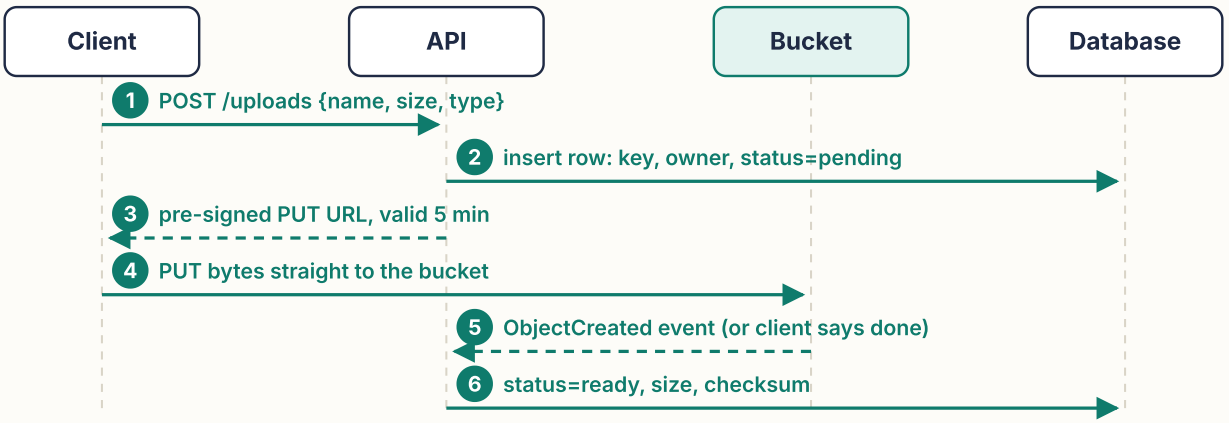


Figure 3. The API only issues permission and records facts; the bytes go straight from the client to the bucket (lessons 4–5).

WORKED EXAMPLE: STORING USER UPLOADS IN S3 OR AZURE BLOB

The client asks the API for an upload slot. The API authenticates the user, checks the declared size against their quota, chooses the key itself (`uploads/user-42/7f3c9e…`, never the client's file name), writes a *pending* row, and returns a pre-signed PUT URL valid for 5 minutes. The client uploads directly to the bucket, so a 200 MB video never touches the API servers. An `ObjectCreated` event (or a "done" call from the client, verified with a `HEAD`) marks the row *ready* with the real size and checksum.

Downloads mirror this: public files go through a CDN with long cache times; private files get a short-lived signed GET URL (lessons 5, 11).

TRADE-OFFS

- Cheap and unlimited, but no in-place edits, appends or locks: workloads that modify files need block or file storage (lesson 2).
- Higher per-request latency (tens of milliseconds to first byte) than a local disk; great for throughput, poor for many tiny random reads.
- Requests cost money: billions of tiny objects can cost more in PUT/GET fees than in storage. Batch small items into larger objects.
- No queries over metadata: you need a database (or an inventory report) to answer "which files belong to whom".

COMMON MISTAKES

- Making a bucket public to "just serve files"; keep it private (S3 Block Public Access) and use a CDN or signed URLs.
- Using client-supplied file names as keys: collisions, overwrites and path tricks. Generate keys on the server.
- Treating eleven nines as a backup: deletes and overwrites by your own code are durable too. Turn on versioning.
- Designing "rename folder" or "list everything owned by X" around LIST calls instead of the metadata database.

8 Key terms

Bucket / container: namespace for objects with its own policy and settings

Key: the object's full name; the namespace is flat

Object: immutable data + metadata, replaced as a whole

Read-after-write consistency: a successful write is visible to every later read and list

Durability: probability data is not lost (S3: eleven nines)

Storage class / tier: price-versus-access option for an object

9 Interview questions

Q1. Why use object storage instead of a database for files? BASIC

It's much cheaper per GB, scales almost without limit, is extremely durable (eleven nines on S3), and serves large files directly or via a CDN without loading your database or app servers.

Q2. What does object storage give up? BASIC

In-place edits, appends, locks and transactions across objects, and any querying beyond key and prefix; so keep the metadata (owner, file name, key, size) in your database.

Q3. Is S3 eventually consistent? BASIC

Not any more: since December 2020 S3 offers strong read-after-write consistency for PUT, DELETE and LIST. Caches and cross-region replicas in front of or behind it are still eventually consistent.

Q4. Users want to rename a folder containing a million files. How do you design for that? SCENARIO

Don't encode folders in object keys: store a folder tree in the database and use opaque keys, so a rename updates one row. With key-based folders it is a million COPY + DELETE calls.

Q5. Two clients upload to the same key at the same moment. What happens? SCENARIO

The last write to complete wins and the other is silently replaced. Use unique keys per upload, or conditional writes (If-None-Match / If-Match on S3, ETag conditions or leases on Azure).

Q6. Your app stores millions of 2 KB JSON events in S3 and the bill is mostly requests. What would you change? SCENARIO

Batch events into larger objects (e.g. one compressed file per minute per partition), since per-request fees and minimum billable sizes dominate for tiny objects.

Q7. Eleven nines durability: so we don't need backups? TRICKY

Durability protects against hardware loss, not against deletes, overwrites, ransomware or a compromised account; you still need versioning, Object Lock and copies in another account or region.

Q8. Durability vs availability: S3 Standard is 11 nines and 99.99%. Why so different? TRICKY

Durability is about never losing bytes (redundant fragments across zones); availability is whether a request succeeds right now, which also depends on networks and front ends and can dip without any data loss.

Next: SD-O-002 · Block vs File Storage backendroadmap.com